

Instructions for Lab Submission

1. File Naming and Submission:

- Submit your solution as a **ZIP file** named `lab<number>.<ERP>.zip`, where `<ERP>` is your ERP number. For example, if your ERP number is 12345, the file should be named `lab1_12345.zip`.
- Ensure that the ZIP file contains:
 - C++ source files named as `task<number>.cpp` (e.g., `task1.cpp`).
 - No unnecessary files (such as executables, temporary files, or editor configs).
- Do not submit **TEXT(.txt)** or **EXE(.exe)** files.

2. Code Organization:

- Write each task in its own `.cpp` file (as specified above).
- Ensure that your code is modular and logically separated into functions when specified.

3. Code Neatness:

- Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
- Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
- Add clear explanations where required (or specified)** to make the logic of your program easy to follow.

4. Code Compilation and Functionality:

- Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
- Double-check that the functionality of the program meets the requirements outlined in the task.

5. Additional Instructions:

- Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
- Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
- If you encounter any issues or require clarifications, reach out before the submission deadline.

6. Deadline:

- Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted.
- Double-check that the zip file is correctly named and contains all necessary files before submitting.

7. Plagiarism Policy:

- The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

if-else Statements

An **if-else** statement is used when a program needs to make a decision based on a condition.

If the condition is true, the statements inside the **if** block are executed. Otherwise, the statements inside the **else** block are executed.

```
int age = 20;

if (age >= 18) {
    cout << "Adult";
}
else {
    cout << "Minor";
}
```

Multiple conditions can also be checked using **else if**.

```
int marks = 75;

if (marks >= 80) {
    cout << "A";
}
else if (marks >= 70) {
    cout << "B";
}
else {
    cout << "C";
}
```

Nested if-else Statements

A nested **if-else** occurs when an **if** or **else** statement contains another **if-else** statement.

It is useful when one condition should only be checked after another condition has already been satisfied.

```
int age = 20;
int idValid = 1;

if (age >= 18) {
    if (idValid == 1) {
        cout << "Access Granted";
    }
    else {
        cout << "Invalid ID";
    }
}
else {
    cout << "Age requirement not met";
}
```

In this example, the ID is checked only if the age condition is satisfied.

Pre-Increment and Post-Increment Operators

The increment operator **++** increases the value of a variable by 1.

There are two forms:

- **Pre-increment:** **++x**
- **Post-increment:** **x++**

When used by itself, both increase the variable by 1.

```
int x = 5;

++x;    // x becomes 6
x++;    // x becomes 7
```

The difference becomes important when the increment operator is used inside another expression.

Pre-increment increases the value first and then uses it.

```
int x = 5;
int y = ++x;

// x = 6
// y = 6
```

Post-increment uses the current value first and then increases it.

```
int x = 5;
int y = x++;

// y = 5
// x = 6
```

Similarly, `--x` and `x--` can be used to decrease a value by 1.

switch Statement

A `switch` statement is useful when one variable or expression can have several fixed possible values.

```
int choice = 2;

switch (choice) {
    case 1:
        cout << "Option 1";
        break;

    case 2:
        cout << "Option 2";
        break;

    case 3:
        cout << "Option 3";
        break;

    default:
        cout << "Invalid option";
}
```

Each `case` represents a possible value.

The `default` block is executed when none of the cases match.

The break Statement in switch

The `break` statement stops the execution of the current `switch` case and exits the `switch` statement.

Without `break`, execution continues into the following cases. This is called **fall-through**.

For example:

```
int number = 1;

switch (number) {
```

```
case 1:
    cout << "One ";
case 2:
    cout << "Two ";
case 3:
    cout << "Three ";
}
```

The output will be:

```
One Two Three
```

This happens because there are no `break` statements.

Using `break` prevents this behavior:

```
switch (number) {
    case 1:
        cout << "One";
        break;

    case 2:
        cout << "Two";
        break;

    case 3:
        cout << "Three";
        break;
}
```

for Loop

A `for` loop is used to repeat a block of code a known number of times.

The general form is:

```
for (initialization; condition; update) {
    // statements
}
```

The three parts are:

- **Initialization:** runs once before the loop starts.
- **Condition:** checked before every iteration.
- **Update:** runs after every iteration.

Example:

```
for (int i = 1; i <= 5; i++) {
    cout << i << endl;
}
```

Output:

```
1
2
3
4
5
```

For this lab, only a **single loop** is required where specified. Nested loops are not required.

A loop can also decrease its counter:

```
for (int i = 5; i >= 1; i--) {
    cout << i << endl;
}
```

ANSI Escape Codes

ANSI escape codes can be used to change how text appears in a terminal.

An ANSI escape sequence usually begins with:

```
\033[
```

and is followed by a formatting or color code.

The code:

```
\033[0m
```

resets the terminal formatting back to its default appearance.

ANSI Text Color Codes

ANSI color codes can be used to display terminal text in different colors.

The general form is:

```
\033[<code>m
```

For example, `\033[31m` changes the foreground text color to red.

Color	ANSI Code	Color Preview
Black	<code>\033[30m</code>	
Red	<code>\033[31m</code>	
Green	<code>\033[32m</code>	
Yellow	<code>\033[33m</code>	
Blue	<code>\033[34m</code>	
Magenta	<code>\033[35m</code>	
Cyan	<code>\033[36m</code>	
White	<code>\033[37m</code>	

Example:

```
cout << "\033[31m";
cout << "This text is red." << endl;
cout << "\033[0m";
```

The code `\033[31m` changes the text color to red, while `\033[0m` resets the terminal formatting to its default.

ANSI Background Color Codes

ANSI escape codes can also be used to change the background color of text.

Color	ANSI Code	Color Preview
Black	\033[40m	
Red	\033[41m	
Green	\033[42m	
Yellow	\033[43m	
Blue	\033[44m	
Magenta	\033[45m	
Cyan	\033[46m	
White	\033[47m	

Example:

```
cout << "\033[41mRed Background\033[0m" << endl;
cout << "\033[42mGreen Background\033[0m" << endl;
```

ANSI Formatting Codes

ANSI escape codes can also be used to change the style or formatting of terminal text.

Formatting	ANSI Code	Example
Bold	\033[1m	Bold Text
Italic	\033[3m	<i>Italic Text</i>
Underline	\033[4m	<u>Underlined Text</u>
Blinking	\033[5m	Blinking Text
Inverted Colors	\033[7m	Inverted Text
Reset	\033[0m	Normal Text

Example:

```
cout << "\033[1mBold Text\033[0m" << endl;
cout << "\033[4mUnderlined Text\033[0m" << endl;
cout << "\033[3mItalic Text\033[0m" << endl;
cout << "\033[5mBlinking Text\033[0m" << endl;
cout << "\033[7mInverted Colors\033[0m" << endl;
```

The reset code \033[0m returns the text to its normal terminal appearance.

Note: Some ANSI effects depend on the terminal. For example, blinking text may not be supported by every terminal. Since a PDF is static, the blinking effect cannot be demonstrated directly in the table.

Pseudo-Random Number Generation

C++ provides the `rand()` function for generating pseudo-random numbers.

To use `rand()` and `srand()`, include:

```
#include <cstdlib>
```

Example:

```
int number = rand();  
  
cout << number;
```

The values generated by `rand()` are called **pseudo-random** because they are produced using a mathematical sequence.

Using `srand()` and `time()`

If `rand()` is used without setting a new seed, the program may produce the same sequence of numbers each time it runs.

The `srand()` function is used to set the starting seed for `rand()`.

A common approach is to use the current time as the seed.

For this, include both:

```
#include <cstdlib>  
#include <ctime>
```

Then write:

```
srand(time(0));
```

This should normally be done **once** near the beginning of the program.

Complete example:

```
#include <iostream>  
#include <cstdlib>  
#include <ctime>  
using namespace std;  
  
int main() {  
  
    srand(time(0));  
  
    int number = rand();  
  
    cout << number;  
  
    return 0;  
}
```

Generating a Random Number within a Range

The modulus operator can be used with `rand()` to restrict the generated value to a particular range.

For a random number from 0 to 9:

```
int number = rand() % 10;
```

For a random number from 1 to 10:

```
int number = rand() % 10 + 1;
```

In general, for a value from 1 to N:

```
int number = rand() % N + 1;
```

For example, simulating a six-sided dice:

```
int dice = rand() % 6 + 1;

cout << "Dice value: " << dice;
```

A complete example is:

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {

    srand(time(0));

    int randomNumber = rand() % 100 + 1;

    cout << "Random number: " << randomNumber;

    return 0;
}
```

Basic String Operations

A `string` is used to store a sequence of characters.

To use strings, include:

```
#include <string>
```

A string can be declared and assigned a value as follows:

```
string city = "Karachi";
string course = "Programming";
```

Strings can also be taken as input from the user.

```
string name;

cout << "Enter your name: ";
cin >> name;
```

String Indexing

Each character inside a string has a position called an **index**.

String indexing starts from `0`.

For example:

```
string word = "HELLO";

cout << word[0] << endl;
cout << word[2] << endl;
```

Output:

```
H
L
```

For the string `"HELLO"`:

- `word[0]` is H
- `word[1]` is E
- `word[2]` is L
- `word[3]` is L
- `word[4]` is 0

Finding the Length of a String

The `length()` operation gives the number of characters stored inside a string.

```
string animal = "Tiger";  
cout << animal.length();
```

Output:

```
5
```

Since indexing starts from 0, the index of the final character is always one less than the length of the string.

For example:

```
string animal = "Tiger";  
char finalLetter = animal[animal.length() - 1];  
cout << finalLetter;
```

Output:

```
r
```

Comparing Characters

Characters taken from a string can be compared using comparison operators.

For example:

```
string word = "LEVEL";  
char second = word[1];  
char fourth = word[3];  
  
if (second == fourth) {  
    cout << "Characters are equal";  
}  
else {  
    cout << "Characters are different";  
}
```

Here, the program compares two selected positions inside the string.

String Concatenation

String concatenation means joining strings together.

The `+` operator can be used to combine strings.

```
string firstName = "Ali";  
string lastName = "Khan";  
  
string fullName = firstName + " " + lastName;  
  
cout << fullName;
```

Output:

```
Ali Khan
```

Several pieces of text can also be joined together.

```
string item = "Laptop";  
string message = string("Selected Item: ") + item + " - Available";  
  
cout << message;
```

Output:

```
Selected Item: Laptop - Available
```

Lab Tasks

Task 1: Counter Control Console

A small event management system keeps track of the number of visitors entering a hall. The programmer wants to understand how pre-increment and post-increment affect values during execution.

Problem Description

Write a C++ program that asks the user to enter an initial visitor count.

Using the entered value, perform a sequence of operations using both **pre-increment** and **post-increment** operators.

Store the result of each operation in separate integer variables and display the values of both the original counter and the stored result after each step.

Perform the following operations in order:

1. Use post-increment on the counter and store the result.
2. Use pre-increment on the counter and store the result.
3. Use post-increment once again and store the result.

Requirements

- Take one integer input from the user.
- Use both pre-increment `++x` and post-increment `x++`.
- Store the result of each increment operation in a separate variable.
- Display the counter value and stored value after each operation.
- Do not use loops or functions.

Test Case

```
Input:
Enter initial visitor count: 5

Output:
After post-increment:
Stored Value: 5
Counter Value: 6

After pre-increment:
Stored Value: 7
Counter Value: 7

After post-increment:
Stored Value: 7
Counter Value: 8
```

Output

```
Enter initial visitor count: 5

After post-increment:
Stored Value: 5
Counter Value: 6

After pre-increment:
Stored Value: 7
Counter Value: 7

After post-increment:
Stored Value: 7
Counter Value: 8
```

Figure 1: Task 1 Output

Task 2: Campus Service Kiosk

A university has installed a small digital kiosk that allows students to select one of several campus services.

Problem Description

Write a C++ program that displays the following menu:

- 1 – Library
- 2 – Cafeteria
- 3 – Computer Lab
- 4 – Student Office

Ask the user to enter a choice from 1 to 4.

Use a **switch statement** to display an appropriate message for the selected service.

If the user enters any other value, display an invalid choice message.

Requirements

- Take one integer choice as input.
- Use a **switch** statement.
- Use separate **case** labels for all four options.
- Use **break** after each case.

- Use a `default` case for invalid input.
- Do not use `if-else` for selecting the service.

Test Case

```
Input:
1. Library
2. Cafeteria
3. Computer Lab
4. Student Office

Enter your choice: 3

Output:
Computer Lab selected.
```

Output

```
1. Library
2. Cafeteria
3. Computer Lab
4. Student Office

Enter your choice: 3
Computer Lab selected.
```

Figure 2: Task 2 Output

Task 3: Lucky Token Generator.....

During a university event, each participant receives a randomly generated lucky token number.

Problem Description

Write a C++ program that generates a random token number between **100 and 999**.

Use `srand()`, `rand()`, and `time()` so that the generated number can change each time the program is executed.

After generating the number, display it as the participant's lucky token.

Requirements

- Include the `<cstdlib>` and `<ctime>` libraries.
- Use `srand(time(0))` to seed the random number generator.

- Use `rand()` to generate the random value.
- Generate a number in the range **100 to 999**.
- Display the generated token number.
- Do not use loops or functions.

Sample Output

```
Lucky Token Generator
Your token number is: 483
```

Output

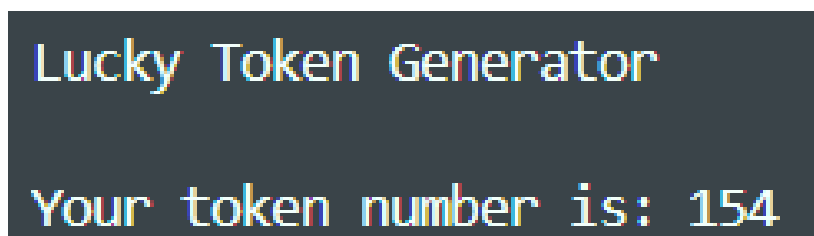


Figure 3: Task 3 Output

Task 4: The Scholarship Committee

The financial aid office is drowning. Applications are piled on every desk, the printer has jammed twice, and the clerk still has a long queue outside.

Problem Description

The committee has one policy and no patience. A student walks in, gives their **CGPA** and **attendance percentage**, and the clerk must announce the scholarship on the spot.

The committee insists the result appear in **colour**, because nobody in that office has the eyes to read plain text anymore.

The committee's policy is fixed:

- If the CGPA is **3.5 or above**, the student receives a **90%** scholarship, displayed in **green**.
- If the CGPA is **3.0 or above but below 3.5**, attendance is checked:
 - attendance **75% or above** → **80%** scholarship in **blue**;
 - otherwise → **70%** scholarship in **yellow**.
- If the CGPA is **2.5 or above but below 3.0**, attendance is checked:
 - attendance **75% or above** → **60%** scholarship in **cyan**;
 - otherwise → **50%** scholarship in **magenta**.
- If the CGPA is **below 2.5**, the student is **Not Eligible**, displayed in **red**.

Requirements

- Use one main `if-else` statement, and make every remaining decision with nested `if-else` statements to reach exactly one verdict.

- Display the student's scholarship percentage in the matching **ANSI colour** and reset the colour afterward.
- **Do not use** `else if`, logical operators, `switch`, loops, arrays, functions, or ternary operators.

Output

```
Enter CGPA: 3.56
Enter attendance percentage: 89
90% Scholarship
```

Figure 4: Task 4 Output

Task 5: The Late Arrival Crisis.....

The professor has had enough. Three weeks of students strolling in long after the lecture has started, and today he finally snaps.

Problem Description

There are N students on the roll. He sets a late limit of L minutes and a threshold of M students. Anyone past L minutes is marked late.

At the end of the hour he does not want names, excuses, or explanations — he wants one verdict, and only one.

Requirements

- Read N , M and L , then read N arrival times using a single `for` loop.
- Print `Class is Cancelled` if more than M students were late, otherwise print `Class is Not Cancelled`.
- **Print nothing else** — no per-student output.
- **Do not use** arrays, functions, or nested loops.

Output

```
Enter the number of students (N), the maximum allowed late students (M), and the latest arrival time (L): 5 3 4
Enter the arrival time for student 1: 2
Enter the arrival time for student 2: 3
Enter the arrival time for student 3: 4
Enter the arrival time for student 4: 5
Enter the arrival time for student 5: 6
Class is Not Cancelled
```

Figure 5: Task 5 Output

Task 6: The Secret Code Analyzer.....

The examination cell has grown paranoid about paper leaks. Every question paper is stamped with a secret code, and the cell has one habit: the code always begins and ends with the same character.

Problem Description

Nobody there wants to check this by hand any more.

The system must read the code, report its first and last characters, confirm whether they match, and stamp the whole thing between `[Start]` and `[End]`.

Requirements

- Store the code in a **string**, extract the first and last characters using **indexing**, check whether they are the same, and build the verification line with **string concatenation**.
- Display the first character, the last character, the match result, and the bracketed verification line.
- **Do not use** loops, arrays, or functions.

Output

```
Enter secret code: Abdullah
First Character: A
Last Character: h
Match Result: No Match
Verification Line: [Start]Abdullah[End]
```

Figure 6: Task 6 Output